

HTTP

HYPERTEXT TRANSFER PROTOCOL



HYPERTEXT TRANSFER PROTOCOL

- Protocollo di livello di **applicazione** nello stack di protocolli Internet.
- Inventato da **Tim Berners-Lee @ CERN 1989-1991**.
- La variante sicura è chiamata **HTTPS**.
- HTTP/3 pubblicato nel 2022.

Versione	Anno di introduzione	Stato attuale
HTTP/0.9	1991	Obsoleto
HTTP/1.0	1996	Obsoleto
HTTP/1.1	1997	Standard
HTTP/2	2015	Standard
HTTP/3	2022	Standard



CLIENT/SERVER

CLIENT

User Agent (UA): qualsiasi programma client che avvia una richiesta (es. browser web, app mobile, spider, elettrodomestici,...)

SERVER

Origin Server (O): un programma che può originare risposte autorevoli per una data risorsa (es. sito web, telecamera per il traffico, macchine da ufficio, piattaforme di video-on-demand,...)



RICHIESTA/RISPOSTA

Richiesta

>

UA ===== 0

<

Risposta



ESEMPIO - RICHIESTA

- **Linea di Richiesta** (metodo-HTTP URI versione-protocollo)
- **Campi di Intestazione della Richiesta**
- **Corpo del Messaggio** (opzionale)

```
GET /hello.txt HTTP/1.1  
User-Agent: curl/7.64.1  
Host: [www.example.com](https://www.example.com)  
Accept-Language: en, it
```



ESEMPIO - RISPOSTA

- **Stato di Completamento** (riguardo alla richiesta)
- **Contenuto** (potrebbe contenere)

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Accept-Ranges: bytes
Content-Length: 51
Vary: Accept-Encoding
Content-Type: text/plain
Hello World! My content includes a trailing CRLF.
```



Catena di connessioni

```
>           >           >           >
UA ===== A ===== B ===== C
===== 0
<           <           <           <
```

Es. Proxy, gateway, tunnel.



CACHE

Un archivio di messaggi di risposta precedenti.

```
>          >
UA ===== A ===== B --- C --- O
<          <
```

- deve dichiarare una risposta come **cacheable** (memorizzabile nella cache)
- i **proxy** possono memorizzare le risposte nella cache
- i **tunnel** non possono memorizzare le risposte nella cache



METODI HTTP



METODI HTTP

- **GET**
- **HEAD**
- **POST**
- **PUT**
- **DELETE**
- **CONNECT**
- **OPTIONS**
- **TRACE**
- **PATCH**



PROPRIETÀ DEI METODI

SAFE (SICURO)

Un metodo che non ha effetti collaterali sulla risorsa, cioè è '**sola lettura**'. Può tuttavia cambiare lo stato del server in altri modi (es. log).

GET, HEAD, OPTIONS e **TRACE** sono sicuri.

IDEMPOTENT (IDEMPOTENTE)

Richieste identiche multiple con quel metodo hanno lo stesso effetto di una singola richiesta. **PUT** e **DELETE** sono idempotenti.

CACHEABLE (MEMORIZZABILE NELLA CACHE)

Metodi che possono consentire a una cache di archiviare e utilizzare una risposta.

GET, HEAD e **POST** (a determinate condizioni) sono memorizzabili nella cache.



PUT

- **Crea una nuova risorsa**, specificandola nella richiesta.
- **Sostituisce una risorsa** (quando l'URI esiste, la sovrascrive con la rappresentazione nel payload PUT).

```
PUT /course-descriptions/web-and-software-architecture
```

- **IDEMPOTENTE**: qualsiasi richiesta PUT identica successiva non modifica la risorsa.
- **Non** è né sicuro (**safe**) né memorizzabile nella cache (**cacheable**).



GET

- **Richiede una rappresentazione dello stato di una risorsa.**

```
GET /course-descriptions/web-and-software-architecture
```

- **SAFE (Sicuro):** nessuna modifica alla risorsa.
- **CACHEABLE (Memorizzabile nella cache):** la risposta può essere memorizzata nella cache da un intermediario e riutilizzata senza richiederla a O; possono essere specificate condizioni (es. data di scadenza, ecc.).
- **Non** è idempotente.



POST

- **Crea o modifica un subordinato della risorsa indicata nell'URI.** L'URI identifica la risorsa che gestirà la richiesta.

```
POST /announcements/  
POST /announcements/{id}/comments/  
POST /users/{id}/email
```

- L'azione **potrebbe non** comportare una nuova risorsa.
- La risposta può essere memorizzabile nella cache se specificato.
- **Non** è sicuro (**safe**).
- **Non** è idempotente! Es. il primo esempio sopra aggiunge molti nuovi annunci identici se richiesto più volte.



DELETE

- Richiede che il server di origine rimuova l'associazione tra la risorsa target e la sua funzionalità attuale.

```
DELETE /courses/web-and-software-architecture
```

- **IDEMPOTENT**: l'eliminazione di una risorsa già eliminata non produce alcun nuovo effetto.
- **Non** è né sicuro (**safe**) né memorizzabile nella cache (**cacheable**).



ALTRI METODI

Metodo	Descrizione
HEAD	Come GET , ma non trasferisce il contenuto della risposta.
CONNECT	Stabilisce un tunnel verso il server identificato dalla risorsa target.
OPTIONS	Descrive le opzioni di comunicazione per la risorsa target.
TRACE	Esegue un test di loop-back del messaggio lungo il percorso verso la risorsa target.



Metodo di richiesta	RFC	rich. payload	resp. payload	Safe	Idempotente	Cacheable
GET	<u>RFC 7231</u>	Opzionale	Sì	Sì	Sì	Sì
HEAD	<u>RFC 7231</u>	Opzionale	No	Sì	Sì	Sì
POST	<u>RFC 7231</u>	Sì	Sì	No	No	Sì
PUT	<u>RFC 7231</u>	Sì	Sì	No	Sì	No
DELETE	<u>RFC 7231</u>	Opzionale	Sì	No	Sì	No
CONNECT	<u>RFC 7231</u>	Opzionale	Sì	No	No	No
OPTIONS	<u>RFC 7231</u>	Opzionale	Sì	Sì	Sì	No
TRACE	<u>RFC 7231</u>	No	Sì	Sì	Sì	No
PATCH	<u>RFC 5789</u>	Sì	Sì	No	No	No



CODICI DI STATO DELLA RISPOSTA

Es. HTTP/1.1 200 OK

- Descrive il risultato della richiesta e la semantica della risposta.
- Se la richiesta ha avuto successo.
- Quale contenuto è allegato (se presente).
- Numero a tre cifre nell'intervallo 100-599.



CLASSI DI CODICI DI STATO

Codice di stato	Descrizione
1xx (Informational)	La richiesta è stata ricevuta, processo in corso.
2xx (Successful)	La richiesta è stata ricevuta, compresa e accettata con successo.
3xx (Redirection)	Sono necessarie ulteriori azioni per completare la richiesta.
4xx (Client Error)	La richiesta contiene sintassi errata o non può essere soddisfatta.
5xx (Server Error)	Il server non è riuscito a soddisfare una richiesta apparentemente valida.



CLASSI DI CODICI DI STATO

Codice di stato	Descrizione
1xx (Informational)	Hold on
2xx (Successful)	Here you go
3xx (Redirection)	Go away
4xx (Client Error)	You fucked up
5xx (Server Error)	I fucked up



2XX SUCCESSO

- **200 OK**
 - In una richiesta **GET**, la risposta conterrà un'entità corrispondente alla risorsa richiesta; in una richiesta **POST**, la risposta conterrà un'entità che descrive o contiene il risultato dell'azione.
- **201 Created**
 - La richiesta è stata soddisfatta, risultando nella **creazione di una nuova risorsa**.
- **204 No Content**
 - Il server ha elaborato con successo la richiesta e **non sta restituendo alcun contenuto**.



3XX REINDIRIZZAMENTO

- **301 Moved Permanently**
 - Questa e tutte le richieste future dovrebbero essere indirizzate all'URI fornito.
- **302 Found** (Precedentemente "Moved temporarily")
 - Guarda un'altra URL.



4XX ERRORI CLIENT

- **400 Bad Request**
 - Apparente errore del client.
- **401 Unauthorized**
 - È richiesta l'autenticazione.
- **403 Forbidden**
 - La richiesta conteneva dati validi ed è stata compresa dal server, ma l'azione è proibita.
- **404 Not Found**
 - La risorsa non è stata trovata ma potrebbe essere disponibile in futuro.
- **405 Method Not Allowed**
 - Il metodo di richiesta non è supportato; ad esempio, una richiesta PUT su una risorsa di sola lettura.



5XX ERRORI SERVER

- **500 Internal Server Error**
 - Condizione inaspettata riscontrata.
- **501 Not Implemented**
 - Metodo di richiesta non riconosciuto, oppure il server non ha la capacità di soddisfare la richiesta.
- **502 Bad Gateway**
 - Un gateway o proxy ha ricevuto una risposta non valida dal server a monte.
- **503 Service Unavailable**
 - Server sovraccarico o inattivo per manutenzione (temporaneo).
- **504 Gateway Timeout**
 - Il server non ha ricevuto una risposta tempestiva dal server a monte.



API PUBBLICHE

API	URL Esempio
SWAPI (Star Wars API)	<code>https://swapi.dev/api/people/1/</code>
JSONPlaceholder	<code>https://jsonplaceholder.typicode.com/posts/1</code>
Nationalize	<code>https://api.nationalize.io/?name=pietro</code>
Chuck Norris	<code>https://api.chucknorris.io/jokes/random</code>
PokeAPI	<code>https://pokeapi.co/api/v2/pokemon/bulbasaur</code>
http.cat	<code>https://http.cat/status/510.jpeg</code>



cURL

Metodo	Scopo	Sintassi cURL
GET (Default)	Scarica il contenuto della risorsa.	<code>curl https://swapi.dev/api/people/1/</code>
HEAD	Richiede solo le intestazioni (Header), non il corpo.	<code>curl -I https://swapi.dev/api/people/1/</code>
POST	Invia dati per creare una nuova risorsa.	<code>curl -X POST -H "Content-Type: application/json" -d '{"title": "Test"}' https://jsonplaceholder.typicode.com/posts</code>



cURL (CONT'D)

Metodo	Scopo	Sintassi cURL
PUT	Invia dati per sostituire una risorsa esistente (Idempotente).	<pre>curl -X PUT -H "Content-Type: application/json" -d '{"id": 1, "title": "New Title"}' https://jsonplaceholder.typicode.com/posts/1</pre>
DELETE	Elimina la risorsa.	<pre>curl -X DELETE https://jsonplaceholder.typicode.com/posts/1</pre>
HEADERS	Visualizza tutte le intestazioni (opzione per GET).	<pre>curl -i https://swapi.dev/api/people/1/</pre>



ESERCIZIO: CRAWLER

- Vogliamo trovare quale fosse l'aspetto del sito del Dipartimento di Informatica **ogni anno**, focalizzandoci sullo snapshot più vicino al **15 Agosto**.
- L'API della Wayback Machine dà accesso allo snapshot più vicino ad una data `https://web.archive.org/web/{YYYY}{MMDD}id_{dominio}`
- L'API CDX della Wayback Machine è un servizio di metadati che risponde a questo formato di URL:
`http://web.archive.org/cdx/search/cdx?url=<dominio>&from=<anno_inizio>&to=<anno_fine>&matchType=host&output=json`
- Dobbiamo seguire i redirect quando presenti



LINK

- [https://en.wikipedia.org/wiki/Hypertext Transfer Protocol](https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol)
- [https://en.wikipedia.org/wiki/List of HTTP header fields](https://en.wikipedia.org/wiki/List_of_HTTP_header_fields)
- [https://en.wikipedia.org/wiki/List of HTTP status codes](https://en.wikipedia.org/wiki/List_of_HTTP_status_codes)
- <https://www.rfc-editor.org/rfc/rfc7231>
- <https://www.rfc-editor.org/rfc/rfc9110.html>
- <https://http.cat>

